

Seminar 2

Last time: start from the **theorem**, leave holes `?_`, **fill them in**.

Today: the same loop, but you get to **name the moves**. Nothing new is being proved — the same terms are being written a different way.

last week: implS

```
theorem implS {A B C : Prop} :  
  (A → B → C) → (A → B) → A → C :=  
  fun f g a => f a (?_)
```

applied `f a` — the hole must be a `B`

```
f : A → B → C  
g : A → B  
a : A  
⊢ B
```

You wrote a term. But you did not write it in one go — you wrote a hole, read what Lean said, and refined.

You were already running a loop

```
graph LR; A[look at the hole's type] --> B[pick a move]; B --> C[write a smaller hole]; C --> A;
```

look at the hole's type → pick a move → write a smaller hole

The **file** was static. The **process** was not.

Lean kept the state — the hypotheses in scope and the type of the hole — and handed it back to you after every edit.

Tactics are that loop, with the moves given names.

The same proof, twice

```
theorem iff_and_congr (hab : A ↔ B) (hcd : C ↔ D) : A ∧ C ↔ B ∧ D :=  
  { fun h1 => { hab.mp h1.left , hcd.mp h1.right } ,  
    fun h2 => { hab.mpr h2.left , hcd.mpr h2.right } }
```

```
theorem iff_and_congr (hab : A ↔ B) (hcd : C ↔ D) : A ∧ C ↔ B ∧ D := by  
  constructor  
  · intro h  
    exact {hab.mp h.left, hcd.mp h.right}  
  · intro h  
    exact {hab.mpr h.left, hcd.mpr h.right}
```

And you can work forwards

"From `hA` and `hr` I get `role B = .knight`. Put it aside; I will use it in a moment."

```
have hb : role B = .knight := hA.mp hr
```

```
hA : role A = .knight ↔ role B = .knight  
hB : role B = .knight ↔ role A = .knave  
hr : role A = .knight  
hb : role B = .knight      -- new  
⊢ False
```

Underneath, it is a term

```
theorem implModusPonens {A B : Prop} : (A → B) ∧ A → B := by
  intro h
  exact h.left h.right

#print implModusPonens
```

```
theorem Seminar02.implModusPonens : ∀ {A B : Prop}, (A → B) ∧ A → B :=
fun {A B} h => h.left h.right
```

But not always the term you'd write

```
theorem andOrDistrib (h : A ∧ (B ∨ C)) : (A ∧ B) ∨ (A ∧ C) := by
  obtain ⟨a, hbc⟩ := h
  cases hbc with
  | inl b => left; exact ⟨a, b⟩
  | inr c => right; exact ⟨a, c⟩
```

```
fun {A B C} h =>
  And.casesOn h fun a hbc =>
    Or.casesOn (motive := fun t => hbc = t → A ∧ B ∨ A ∧ C) hbc
      (fun b h => Or.inl ⟨a, b⟩) (fun c h => Or.inr ⟨a, c⟩) (Eq.refl hbc)
```

The dictionary — using and building

you wrote (hw01)	you can now say
<code>?_</code>	one goal in the goal state
<code>fun x => ?_</code>	<code>intro x</code>
the finished term <code>e</code>	<code>exact e</code>
<code>f ?_ ?_</code>	<code>apply f</code>
<code><?_, ?_></code> , <code>And.intro</code> , <code>Iff.intro</code>	<code>constructor</code>
<code>Or.inl ?_ / Or.inr ?_</code>	<code>left / right</code>
<code>match h with .inl a => ?_ .inr b => ?_</code>	<code>cases h with ... / rcases h with a b</code>
<code>match Classical.em A with ...</code>	<code>by_cases hA : A</code>
<code>fun <a, b> => ?_</code> , <code>h.left</code> , <code>h.right</code>	<code>obtain <a, b> := h</code>
naming a subterm you reuse	<code>have hb : B := ...</code>
<code>False.elim ?_</code>	<code>exfalso</code>

New this week: rewriting and quantifiers

tactic	what it does
<code>cases hr : role x</code>	one goal per constructor of <code>Role</code> ; <code>hr</code> records which
<code>rfl</code>	closes <code>a = b</code> when both sides compute to the same value
<code>rw [h]</code>	<code>h : a = b</code> , so replace <code>a</code> by <code>b</code> in the goal
<code>rw [← h]</code>	the same rewrite, right to left
<code>rw [h] at h'</code>	rewrite inside a hypothesis instead of the goal
<code>intro x</code>	$\forall x, P\ x$ is a function type — the same tactic as for $\rightarrow$
<code>exact ⟨x, prf⟩</code>	$\exists x, P\ x$ is a pair: a witness, and a proof about it
<code>obtain ⟨x, hx⟩ := h</code>	take that pair apart

Round 1 — translate

`code/Course/Seminar02.lean`, round 1. Four of last week's exercises, with your term proof in the comment above each. **Write the tactic proof.**

```
andAssocBack      : A ∧ (B ∧ C) → (A ∧ B) ∧ C
iffNotCongr       : (A ↔ B) → (¬A ↔ ¬B)
noContradiction    : ¬(A ∧ ¬A)
notNotEm           : ¬¬(A ∨ ¬A)
```

andAssocBack

```
theorem andAssocBack {A B C : Prop} (h : A ∧ (B ∧ C)) :  
  (A ∧ B) ∧ C := by  
  obtain ⟨a, b, c⟩ := h  
  exact ⟨⟨a, b⟩, c⟩
```

after `obtain`

```
a : A  
b : B  
c : C  
⊢ (A ∧ B) ∧ C
```

after `exact ⟨⟨a, b⟩, c⟩`

No goals

`obtain` names the pieces once; hw01 reached in with `h.left` and `h.right.left` instead.

iffNotCongr

```
theorem iffNotCongr {A B : Prop} (hab : A ↔ B) : ¬A ↔ ¬B := by
  constructor
  · intro na b
    exact na (hab.mpr b)
  · intro nb a
    exact nb (hab.mp a)
```

after **constructor** — 2 goals

```
⊢ ¬A → ¬B
⊢ ¬B → ¬A
```

after both bullets

No goals

constructor on an $\leftrightarrow$ is the $\langle _, _ \rangle$ you wrote by hand; the dots pick which half you are on.

noContradiction

```
theorem noContradiction {A : Prop} : ¬(A ∧ ¬A) := by
  intro h
  exact h.right h.left
```

after `intro h`

```
h : A ∧ ¬A
⊢ False
-- ¬X is X → False, so the goal
-- of a negation is always False
```

after `exact h.right h.left`

No goals

A negation goal starts with `intro`, like any other function goal. `¬` carries no tactics of its own.

notNotEm

```
theorem notNotEm {A : Prop} :  $\neg\neg(A \vee \neg A)$  := by
  intro h
  ?_
```

```
h :  $\neg(A \vee \neg A)$ 
⊢ False
-- h :  $(A \vee \neg A) \rightarrow \text{False}$ 
-- the only move: feed it an  $A \vee \neg A$ 
```

notNotEm

```
theorem notNotEm {A : Prop} :  $\neg\neg(A \vee \neg A)$  := by
  intro h
  apply h
  right
  ?_
```

`apply h` , then pick a side

```
h :  $\neg(A \vee \neg A)$ 
 $\vdash A \vee \neg A$           -- after apply h
 $\vdash \neg A$               -- after right
-- left would need an A, and
-- we have no A. So: right.
```

notNotEm

```
theorem notNotEm {A : Prop} :  $\neg\neg(A \vee \neg A)$  := by
  intro h
  apply h
  right
  intro a
  apply h
  left
  exact a
```

now we have an **A** — use **h** again

```
a : A
h :  $\neg(A \vee \neg A)$ 
⊢ False      -- after intro a
⊢  $A \vee \neg A$  -- after apply h
⊢ A          -- after left
```

No goals

#print notNotEm

```
theorem Seminar02.notNotEm :  $\forall \{A : \text{Prop}\}, \neg\neg(A \vee \neg A) :=$   
fun {A} h => h (Or.inr fun a => h (Or.inl a))
```

Round 2 — back-translate

Round 2 of the file. Two tactic proofs are **given**.

For each: predict what `#print` shows, write the term yourself underneath, then run it and compare.

```
implModusPonens : (A → B) ∧ A → B          -- intro / exact
andOrDistrib     : A ∧ (B ∨ C) → (A ∧ B) ∨ (A ∧ C)  -- obtain / cases
```

Round 2 — the answers

```
-- implModusPonens: exactly what you would type
fun {A B} h => h.left h.right

-- andOrDistrib: not what you would type
fun {A B C} h =>
  And.casesOn h fun a hbc =>
    Or.casesOn (motive := fun t => hbc = t → A ∧ B ∨ A ∧ C) hbc
      (fun b h => Or.inl ⟨a, b⟩) (fun c h => Or.inr ⟨a, c⟩) (Eq.refl hbc)
```

Round 3 — where the term is not worth writing

```
flipFlipFlip      : flip (flip (flip r)) = flip r      -- cases, rfl
roleChain3        : role A = role B → role B = role C → role A = role C
roleChain3'       : the same, backwards and at a hypothesis
forallImpForall   : (∀ x, P x → Q x) → (∀ x, P x) → ∀ x, Q x
existsSwap        : (∃ x, ∃ y, P x y) → ∃ y, ∃ x, P x y
puzzleMutualAccusation -- last week's puzzle, new encoding
```

flipFlipFlip

```
def Role.flip : Role → Role
| .knight => .knave
| .knave  => .knight
```

```
theorem flipFlipFlip (r : Role) :
  Role.flip (Role.flip (Role.flip r)) = Role.flip r := by
  cases r <;> rfl
```

after `cases r` — 2 goals

```
⊢ flip (flip (flip knight))
  = flip knight
⊢ flip (flip (flip knave))
  = flip knave
-- both sides compute
```

after `rfl` on each

No goals

Two constructors, so `cases r` opens two goals — and `<;>` runs `rfl` on every one of them.

The rewrites

```
theorem roleChain3 (h1 : role A = role B) (h2 : role B = role C) :  
  role A = role C := by  
  rw [h1, h2]
```

```
theorem roleChain3' (h1 : role A = role B) (h2 : role B = role C) :  
  role A = role C := by  
  rw [← h1] at h2  
  exact h2
```

the goal, rewritten twice

```
⊢ role A = role C  
⊢ role B = role C    -- rw [h1]  
⊢ role C = role C    -- rw [h2]
```

No goals

`rw [h]` rewrites the goal; `← h` reverses the direction, `at h'` aims it at a hypothesis.

The quantifiers, with no new tactics

```
theorem forallImpForall {α : Type} (P Q : α → Prop) :  
  (∀ x, P x → Q x) → (∀ x, P x) → ∀ x, Q x := by  
  intro h hp x  
  exact h x (hp x)
```

```
theorem existsSwap {α β : Type} (P : α → β → Prop) :  
  (∃ x, ∃ y, P x y) → ∃ y, ∃ x, P x y := by  
  intro h  
  obtain ⟨x, y, hxy⟩ := h  
  exact ⟨y, x, hxy⟩
```

three `intro` s, one of them a `∀`

```
h   : ∀ x, P x → Q x  
hp  : ∀ x, P x  
x   : α  
⊢ Q x
```

puzzleMutualAccusation

```
theorem puzzleMutualAccusation (A B : Islander)
  (hA : Says A (role B = .knight)) (hB : Says B (role A = .knave)) :
  claim% answerMutualAccusation [A, B] := by
  unfold Says at hA hB
  cases hr : role A with
  | knight =>
    have hna : role A = .knave := hB.mp (hA.mp hr)
    rw [hr] at hna
    cases hna
  | knave =>
    have hka : role A = .knight := hA.mpr (hB.mpr hr)
    rw [hr] at hka
    cases hka
```

The answer is `impossible`, so the goal is `False`. Two constructors, two cases; `rw [hr]` turns each `have` into `knight = knave`, and `cases` on that closes the goal.

The same puzzle, last week

```
-- hw01: an islander was a Prop, and the split came from an axiom
match Classical.em A with
| Or.inl a  => (hB.mp (hA.mp a)) a
| Or.inr na => na (hA.mpr (hB.mpr na))
```

```
-- today: an islander is a value, and the split comes from the data
cases hr : role A with
| knight => ...
| knave  => ...
```

```
#print axioms puzzleMutualAccusation
-- 'Seminar02.puzzleMutualAccusation' does not depend on any axioms
```

The case split now comes from `Role` having exactly two constructors. No excluded middle, no axioms.

What to take away

- A goal **is** a hole; `intro` / `exact` / `apply` are edits to it.
- `#print` shows what your tactics wrote — often not the term you would write.
- `∀` is `→` with a name, `∃` is `λ` with a witness — no new tactics.
- A finite type's case split comes from the **data**, not from `Classical.em`.
- `rw [h]` rewrites the goal; `← h` reverses it, `at h'` aims it elsewhere.
- Everything else today was a **renaming** of something you already did.